

A Beginner's Guide to Linear Classification Algorithms

Written for First-Year Students and Aspiring Data Scientists

Introduction

Welcome to the world of Machine Learning! If you have ever wondered how email systems automatically filter spam, how banks instantly approve or reject credit card applications, or how medical software flags high-risk patients, you are looking at **Classification**.

This guide is designed for complete beginners. If you have never taken a machine learning course, don't worry! We will build everything from the ground up using simple language, physical analogies, step-by-step math, and clean Python code.

What is a Linear Classifier?

Imagine you have a table covered in two types of candies: **Red Cherry Drops** and **Blue Blueberry Chews**. They are mixed up, but generally, the cherry drops are on the left side of the table, and the blueberry chews are on the right.

If you wanted to separate them perfectly, the simplest thing you could do is place a straight **wooden ruler** on the table so that all the red candies are on one side and all the blue candies are on the other.

- **The Ruler** is the **Decision Boundary**.
- Because the ruler is straight, this is a **Linear Classifier**.
- If the table was 3-dimensional (like a box of candies), the ruler would become a flat sheet of paper. In machine learning, this flat boundary is called a **Hyperplane**.

Let's dive into the eight core linear classification algorithms!

1 The Perceptron

1.1 What is it? (The Intuitive Definition)

The Perceptron is the ancestor of modern Deep Learning and Artificial Neural Networks. Created by Frank Rosenblatt in 1958, it is a virtual “neuron” that takes multiple inputs, multiplies them by their importance level (weights), adds them up, and passes them through a switch to make a hard “yes” or “no” decision. It learns by trial and error: when it makes a mistake, it slightly adjusts its parameters.

1.2 The Analogy: The Strict Club Bouncer

Imagine a club bouncer deciding who gets inside. The bouncer has a mental checklist:

1. **Nice shoes?** (Value: x_1)
2. **Polite attitude?** (Value: x_2)

The bouncer values politeness much more than shoes, so they assign a high weight (w_2) to attitude and a small weight (w_1) to shoes. The bouncer adds these scores up. If the total is above a certain baseline score (the **bias**, b), they let you in.

If they accidentally reject a perfectly polite person, the bouncer feels bad and “nudges” their mental rules so they are more welcoming to polite people next time. This nudge is the **learning rule**.

1.3 The Mathematical Concept

The Perceptron calculates a score z :

$$z = w_1x_1 + w_2x_2 + b$$

Where:

- x_1, x_2 are the inputs (features).
- w_1, w_2 are the weights (importance values).
- b is the bias (baseline offset).

It then applies a **Step Function**:

- If $z \geq 0$, predict **Class +1** (Pass/Yes).
- If $z < 0$, predict **Class -1** (Fail/No).

If the prediction $\hat{y}$ does not match the actual target y , we update our weights and bias:

$$\begin{aligned}w_{\text{new}} &= w_{\text{old}} + \eta(y - \hat{y})x \\b_{\text{new}} &= b_{\text{old}} + \eta(y - \hat{y})\end{aligned}$$

Where η (pronounced “eta”) is the **Learning Rate** (how big of a step we take to correct our mistake).

1.4 Step-by-Step Worked Example

1.4.1 The Dataset

We want to predict if a student will **Pass (+1)** or **Fail (-1)** based on **Hours Studied** (x_1).

Student	Hours Studied (x_1)	Actual Result (y)
S1	8.0	+1 (Pass)
S2	2.0	-1 (Fail)

1.4.2 Step-by-Step Calculations

Let's initialize our model with a weight $w_1 = -0.5$, a bias $b = 0.5$, and set our learning rate $\eta = 0.1$.

Step 1: Test the model on Student 1 ($x_1 = 8.0, y = +1$)

1. Compute the score z :

$$z = (w_1 \cdot x_1) + b = (-0.5 \cdot 8.0) + 0.5 = -4.0 + 0.5 = -3.5$$

2. Make a prediction $\hat{y}$:

Since $z = -3.5$ (which is less than 0), the Perceptron predicts $\hat{y} = -1$ (Fail).

3. Check the error:

The actual result is $y = +1$, but we predicted $\hat{y} = -1$. This is a mistake!

4. Update parameters:

$$w_{\text{new}} = w_{\text{old}} + \eta(y - \hat{y})x_1$$

$$w_{\text{new}} = -0.5 + 0.1 \cdot (1 - (-1)) \cdot 8.0$$

$$w_{\text{new}} = -0.5 + 0.1 \cdot (2) \cdot 8.0 = -0.5 + 1.6 = 1.1$$

$$b_{\text{new}} = b_{\text{old}} + \eta(y - \hat{y})$$

$$b_{\text{new}} = 0.5 + 0.1 \cdot (2) = 0.5 + 0.2 = 0.7$$

Our new parameters are $w_1 = 1.1$ and $b = 0.7$. Let's see how they perform on S1 now:

$$z = (1.1 \cdot 8.0) + 0.7 = 8.8 + 0.7 = 9.5 \quad (\text{Since } 9.5 \geq 0, \hat{y} = +1! \text{ Correct!})$$

1.4.3 Plain English Interpretation

The model originally had a negative weight for study hours, meaning "studying more makes you fail." By testing S1, it realized this rule was wrong, updated the weight to a positive number (1.1), and successfully learned that studying is beneficial!

1.5 Python Implementation & Code Walkthrough

```

1 import numpy as np
2
3 class Perceptron:
4     def __init__(self, learning_rate=0.1, epochs=10):
5         self.lr = learning_rate
6         self.epochs = epochs
7         self.w = None
8         self.b = 0.0
9
10    def fit(self, X, y):

```

```

11     # Determine the number of features (columns) in our data
12     num_features = X.shape[1]
13     # Start with all weights set to zero
14     self.w = np.zeros(num_features)
15     self.b = 0.0
16
17     # Loop through the dataset multiple times (epochs)
18     for epoch in range(self.epochs):
19         for idx, x_i in enumerate(X):
20             # Calculate the raw decision score (z = w * x + b)
21             z = np.dot(x_i, self.w) + self.b
22             # Step function: Predict 1 if score >= 0, else predict -1
23             y_pred = 1 if z >= 0 else -1
24
25             # If prediction is wrong, update weights and bias
26             if y_pred != y[idx]:
27                 update = self.lr * (y[idx] - y_pred)
28                 self.w += update * x_i
29                 self.b += update

```

1.5.1 Under the Hood Code Breakdown

- `self.w = np.zeros(num_features)`: We start with a blank slate by setting all feature weights to 0.
- `np.dot(x_i, self.w)`: This performs matrix multiplication to sum up all $(x \cdot w)$ pairs instantly.
- `update = self.lr * (y[idx] - y_pred)`: If prediction equals actual, the difference is 0 and no update happens. If they differ, this calculates the direction and size of the correction.

2 Logistic Regression

2.1 What is it? (The Intuitive Definition)

Despite its confusing name, **Logistic Regression is a Classifier**. Instead of giving a blunt “yes” or “no” answer, it tells you the **probability** (a percentage chance between 0% and 100%) that an item belongs to a certain group.

2.2 The Analogy: The Weather Forecaster

If you ask a robotic friend “Is it going to rain today?”, a Perceptron-style robot will say “Yes” or “No”.

A Logistic Regression robot behaves like a human meteorologist. It looks at the humidity and clouds and says: “*There is an 82% chance of rain today.*” You can then make your own decision: if the chance is over 50%, you pack an umbrella.

2.3 The Mathematical Concept

Logistic Regression uses a mathematical curve called the **Sigmoid Function** (or Logistic Curve) to squash any real number score (z) into a probability range between 0 and 1.

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Where:

- e is Euler’s constant (≈ 2.71828).
- When z is a large positive number, $\sigma(z)$ gets very close to 1 (100% probability).
- When z is a large negative number, $\sigma(z)$ gets very close to 0 (0% probability).
- When $z = 0$, $\sigma(z) = 0.5$ (50% probability).

2.4 Step-by-Step Worked Example

2.4.1 The Dataset

We want to calculate the probability that a student passes based on **Hours Studied** (x_1).

Student	Hours Studied (x_1)	Actual Result (y)
S1	6.0	1 (Pass)

2.4.2 Step-by-Step Calculations

Let’s assume our trained model has a weight $w_1 = 1.5$ and a bias $b = -6.0$.

1. Compute the linear score z :

$$z = w_1 x_1 + b = (1.5 \cdot 6.0) + (-6.0) = 9.0 - 6.0 = 3.0$$

2. Apply the Sigmoid function to find the probability (P):

$$P = \frac{1}{1 + e^{-3.0}}$$

Let's calculate $e^{-3.0}$:

$$e^{-3.0} \approx 2.71828^{-3} \approx 0.0498$$

Now calculate P :

$$P = \frac{1}{1 + 0.0498} = \frac{1}{1.0498} \approx 0.9526$$

3. Apply a decision threshold (usually 0.5):

Since our calculated probability 0.9526 (95.26%) is greater than 0.5, we classify this student as **Pass (1)**.

2.4.3 Plain English Interpretation

The model calculated a raw score of 3.0 for studying 6 hours. When passed through the sigmoid filter, this translates to a **95.26% confidence** that this student will pass the exam.

2.5 Python Implementation & Code Walkthrough

```
1 import numpy as np
2
3 class LogisticRegression:
4     def __init__(self, learning_rate=0.1, iterations=1000):
5         self.lr = learning_rate
6         self.iterations = iterations
7         self.w = None
8         self.b = 0.0
9
10    def _sigmoid(self, z):
11        # Map any value z to a range between 0 and 1
12        return 1 / (1 + np.exp(-z))
13
14    def fit(self, X, y):
15        num_samples, num_features = X.shape
16        self.w = np.zeros(num_features)
17        self.b = 0.0
18
19        # Gradient Descent optimization loop
20        for _ in range(self.iterations):
21            # 1. Compute raw scores for all samples
22            z = np.dot(X, self.w) + self.b
23            # 2. Transform raw scores into probabilities
24            predictions = self._sigmoid(z)
25
26            # 3. Compute how far off our predictions are (gradients)
27            dw = (1 / num_samples) * np.dot(X.T, (predictions - y))
28            db = (1 / num_samples) * np.sum(predictions - y)
29
30            # 4. Slightly adjust weights and bias to decrease prediction
31            error
32            self.w -= self.lr * dw
33            self.b -= self.lr * db
```

2.5.1 Under the Hood Code Breakdown

- `_sigmoid(self, z)`: A helper method that computes the squashing function using NumPy's vectorized exponent tool (`np.exp`).
- `predictions - y`: Measures the prediction error for every sample.
- `self.w -= self.lr * dw`: Updates weights by moving them in the direction that minimizes overall loss (Gradient Descent).

3 Linear Support Vector Machine (Linear SVM)

3.1 What is it? (The Intuitive Definition)

A Linear Support Vector Machine focuses on safety. Instead of just finding *any* line that separates two groups, SVM tries to find the **widest possible road** between them. By keeping the separator as far away from both groups as possible, it reduces the risk of making mistakes on new, unseen data.

3.2 The Analogy: Building a Grand Highway

Imagine you are building a highway separating two towns: the Red Town and the Blue Town.

You don't want a narrow road that hugs the outer houses of the Red Town, because a child playing near those houses might wander onto the asphalt. Instead, you build a wide highway, paving as much empty land as possible between the towns.

The outer houses of both towns that sit directly on the edge of this paved highway zone are the **Support Vectors**. They "support" and define the limits of your highway's **Margin** (width).

3.3 The Mathematical Concept

The goal of SVM is to maximize the physical margin, which mathematically corresponds to **minimizing** the length of our weight vector:

$$\min \frac{1}{2} \|\mathbf{w}\|^2$$

Subject to the constraint that every data point lies on the correct side of the margin boundaries:

$$y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1$$

- If $y_i(\mathbf{w}^T \mathbf{x}_i + b) = 1$, the data point lies directly on the margin boundary. This point is a **Support Vector**.
- If a dataset cannot be cleanly split with a straight line, we use a **Soft-Margin SVM** which allows minor mistakes, balanced by a penalty parameter **C**.

3.4 Step-by-Step Worked Example

3.4.1 The Dataset

We want to classify credit card applicants into **Low Risk (+1)** or **High Risk (-1)** using **Debt Level (x_1)**.

Applicant	Debt Level (x_1)	Actual Status (y)
A1	1.0	+1 (Low Risk)
A2	5.0	-1 (High Risk)

3.4.2 Step-by-Step Calculations

Let's assume our SVM has solved its equations and found:

- Weight $w_1 = -0.5$
- Bias $b = 1.5$

Let's verify if the margin conditions hold for both applicants:

Step 1: Test Applicant 1 ($x_1 = 1.0, y = +1$)

$$y_1(w_1x_1 + b) = (+1) \cdot ((-0.5 \cdot 1.0) + 1.5) = 1 \cdot (-0.5 + 1.5) = 1 \cdot 1.0 = 1.0$$

*Since this equals exactly 1.0, Applicant 1 is a **Support Vector** lying right on the margin edge!*

Step 2: Test Applicant 2 ($x_1 = 5.0, y = -1$)

$$y_2(w_1x_1 + b) = (-1) \cdot ((-0.5 \cdot 5.0) + 1.5) = -1 \cdot (-2.5 + 1.5) = -1 \cdot (-1.0) = 1.0$$

*Applicant 2 also sits exactly on the negative margin edge, making them our second **Support Vector**!*

3.4.3 Plain English Interpretation

The SVM has positioned its decision boundary exactly at Debt Level = 3.0 ($z = -0.5(3.0) + 1.5 = 0$). The safety margins extend to Debt Level 1.0 on one side and Debt Level 5.0 on the other, creating a safe, balanced buffer zone.

3.5 Python Implementation & Code Walkthrough

```
1 from sklearn.svm import SVC
2 import numpy as np
3
4 # 1. Create a Linear Support Vector Classifier
5 # C=1.0 is our soft-margin compromise parameter
6 svm_model = SVC(kernel='linear', C=1.0)
7
8 # Example dataset
9 X = np.array([[1.0], [5.0]])
10 y = np.array([1, -1])
11
12 # 2. Fit the model to our training data
13 svm_model.fit(X, y)
14
15 # 3. Extract the calculated parameters
16 calculated_weight = svm_model.coef_[0]
17 calculated_bias = svm_model.intercept_
18 support_vectors = svm_model.support_vectors_
```

3.5.1 Under the Hood Code Breakdown

- `kernel='linear'`: This tells Scikit-learn not to curve the boundary; keep the separating line flat and straight.
- `C=1.0`: Balances the trade-off. A large C forces a narrow margin to minimize training errors; a small C prioritizes a wider margin, even if it misclassifies a few points.
- `svm_model.support_vectors_`: Directly retrieves the coordinates of the critical margin-edge data points.

4 Fisher's Linear Discriminant Analysis (LDA)

4.1 What is it? (The Intuitive Definition)

Fisher's LDA is a statistical classification technique. Instead of drawing a boundary directly in a crowded room, LDA works by **projection**. It collapses multidimensional data points down onto a single flat line, choosing an angle that keeps different groups as far apart as possible while making sure each group remains tightly packed.

4.2 The Analogy: Shadows on a Screen

Imagine you are holding a flashlight in a dark room. In front of you is a mixed crowd of tall basketball players and shorter gymnast athletes. Behind them is a flat white sheet.

If you shine the light from directly overhead, their shadows on the floor will overlap, making it hard to tell them apart.

If you move the flashlight to side-level, the shadow clusters on the wall will separate clearly. LDA works like finding the perfect angle for your flashlight so that the shadows of both groups are clearly separated and easy to tell apart.

4.3 The Mathematical Concept

LDA maximizes **Fisher's Criterion** (J):

$$J(\mathbf{w}) = \frac{\text{Separation between class means}}{\text{Spread (variance) within each class}} = \frac{\mathbf{w}^T \mathbf{S}_B \mathbf{w}}{\mathbf{w}^T \mathbf{S}_W \mathbf{w}}$$

Where:

- $\mathbf{S}_B$ is the **Between-Class Scatter** (how far apart the group averages are).
- $\mathbf{S}_W$ is the **Within-Class Scatter** (how spread out the points are inside their own groups).
- $\mathbf{w}$ is the optimal projection direction: $\mathbf{w} = \mathbf{S}_W^{-1}(\mathbf{m}_1 - \mathbf{m}_2)$.

4.4 Step-by-Step Worked Example

4.4.1 The Dataset

We want to classify flowers into **Species A (1)** or **Species B (2)** based on **Petal Width** (x_1).

- **Species A data points:** [1.0, 2.0] (Mean $m_1 = 1.5$)
- **Species B data points:** [5.0, 6.0] (Mean $m_2 = 5.5$)

4.4.2 Step-by-Step Calculations

1. Compute the class means:

$$m_1 = 1.5, \quad m_2 = 5.5$$

2. Compute Within-Class Scatter (S_W) sums:

- For Species A: $(1.0 - 1.5)^2 + (2.0 - 1.5)^2 = 0.25 + 0.25 = 0.5$
- For Species B: $(5.0 - 5.5)^2 + (6.0 - 5.5)^2 = 0.25 + 0.25 = 0.5$

- Total $S_W = 0.5 + 0.5 = 1.0$

3. Compute the optimal projection vector w :

$$w = S_W^{-1}(m_2 - m_1) = \frac{1}{1.0} \cdot (5.5 - 1.5) = 4.0$$

4.4.3 Plain English Interpretation

The LDA projection weight is 4.0. This value indicates that projecting the dataset onto a 1D coordinate line scaled by 4 provides the cleanest possible separation between Species A and Species B.

4.5 Python Implementation & Code Walkthrough

```

1 from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
2 import numpy as np
3
4 # 1. Initialize the LDA model
5 lda = LinearDiscriminantAnalysis()
6
7 # Create structured input features and target labels
8 X = np.array([[1.0], [2.0], [5.0], [6.0]])
9 y = np.array([1, 1, 2, 2])
10
11 # 2. Fit the model to discover the projection axis
12 lda.fit(X, y)
13
14 # 3. Predict the class of a new unknown sample
15 new_sample = np.array([[3.5]])
16 prediction = lda.predict(new_sample)

```

4.5.1 Under the Hood Code Breakdown

- `LinearDiscriminantAnalysis()`: Computes covariance matrices internally to find the projection line that minimizes within-group spread.
- `lda.predict([[3.5]])`: Projects the value 3.5 onto the learned axis and determines which class mean shadow it sits closest to.

5 Ridge Classifier

5.1 What is it? (The Intuitive Definition)

The Ridge Classifier is a regularized linear model. It works by setting up a standard classification line, but includes a built-in constraint that keeps the model's weights small. This prevents the model from relying too heavily on any single feature, encouraging it to distribute its focus across multiple indicators to improve stability.

5.2 The Analogy: The Cautious Investor

Imagine you are building an investment model to predict whether a business will **Succeed (+1)** or **Fail (-1)**. You have two indicators: **CEO Experience** and **Office Decoration Quality**.

A simple classifier might notice that highly successful companies often have expensive office decorations, leading it to assign a massive weight to decoration quality.

A **Ridge Classifier** acts like a cautious investor. It says: *"We shouldn't put all our eggs in one basket. I'm going to set a strict limit on our weights so that we don't rely too heavily on office decorations alone."* This forces the model to look at multiple balanced signals, such as CEO experience and actual revenue.

5.3 The Mathematical Concept

The Ridge Classifier sets up a least-squares problem with a penalty term (known as L_2 **Regularization**) that discourages large weight values:

$$\text{Loss} = \sum_{i=1}^N (y_i - (\mathbf{w}^T \mathbf{x}_i + b))^2 + \alpha \sum_{j=1}^n w_j^2$$

Where:

- The first term measures how close our predictions are to the actual values.
- The second term ($\alpha \sum w_j^2$) is the **Ridge Penalty**.
- α (alpha) is a tuning parameter. A higher α forces weights closer to zero, simplifying the model to prevent overfitting.

5.4 Step-by-Step Worked Example

5.4.1 The Dataset

We want to predict if an email is **Spam (+1)** or **Inbox Safe (-1)** using the feature **Contains "Free"** (x_1).

Email	Contains "Free" (x_1)	Actual Type (y)
E1	1.0 (Yes)	+1 (Spam)

5.4.2 Step-by-Step Calculations

Let's assume our current bias $b = 0$, and we compare two different models:

- **Model A (No Penalty):** Uses a massive weight $w_1 = 10.0$ to make an aggressive prediction.
- **Model B (With Ridge Penalty $\alpha = 2.0$):** Balances prediction accuracy with weight size, using $w_1 = 0.8$.

Let's evaluate their total Loss values under $\alpha = 2.0$:

Model A (No Penalty):

- Prediction: $10.0 \cdot 1.0 = 10.0$
- Prediction Error: $(y - \hat{y})^2 = (1 - 10.0)^2 = (-9)^2 = 81.0$
- Penalty: $\alpha \cdot w_1^2 = 2.0 \cdot (10.0)^2 = 2.0 \cdot 100 = 200.0$
- **Total Loss:** $81.0 + 200.0 = 281.0$

Model B (With Ridge Penalty):

- Prediction: $0.8 \cdot 1.0 = 0.8$
- Prediction Error: $(y - \hat{y})^2 = (1 - 0.8)^2 = (0.2)^2 = 0.04$
- Penalty: $\alpha \cdot w_1^2 = 2.0 \cdot (0.8)^2 = 2.0 \cdot 0.64 = 1.28$
- **Total Loss:** $0.04 + 1.28 = 1.32$

5.4.3 Plain English Interpretation

Even though Model A predicted the correct direction, its massive weight (10.0) triggered a huge Ridge penalty. Model B achieved a much lower overall loss (1.32 vs 281.0) by keeping its weight small and controlled, making it a more stable model.

5.5 Python Implementation & Code Walkthrough

```
1 from sklearn.linear_model import RidgeClassifier
2 import numpy as np
3
4 # 1. Initialize the Ridge Classifier
5 # alpha=1.0 is our regularization penalty strength
6 ridge_model = RidgeClassifier(alpha=1.0)
7
8 # Create an example dataset
9 X = np.array([[1.0], [2.0], [0.0]])
10 y = np.array([1, 1, -1])
11
12 # 2. Train the model
13 ridge_model.fit(X, y)
14
15 # 3. Print the scaled weights
16 print("Ridge Weight:", ridge_model.coef_[0])
```

5.5.1 Under the Hood Code Breakdown

- `RidgeClassifier(alpha=1.0)`: Converts target classes into continuous values (+1 and -1) and solves a regularized linear regression problem.
- `ridge_model.coef_`: Displays the weights, which are kept small and balanced by the L_2 regularization penalty.

6 Passive-Aggressive Classifier

6.1 What is it? (The Intuitive Definition)

The Passive-Aggressive Classifier is an online learning algorithm designed to handle data streaming in real-time. It behaves in one of two distinct ways:

- **Passive:** If a new data point is classified correctly with high confidence, the model makes no changes to its weights.
- **Aggressive:** If the prediction is incorrect or lacks confidence, the model immediately and aggressively adjusts its weights to correct the mistake.

6.2 The Analogy: The Easy-Going Trainer

Imagine a driving instructor teaching you how to stay centered in your lane.

- **Passive Mode:** As long as you stay safely in the middle of the lane, the instructor is completely silent and relaxed. They do not micro-manage you.
- **Aggressive Mode:** The second your tire drifts over the white line, the instructor immediately grabs the wheel and firmly pulls the car back into the lane. They react aggressively to correct the error, then go back to being quiet once you are safe.

6.3 The Mathematical Concept

For each incoming sample $\mathbf{x}_t$, the algorithm calculates a loss value (ℓ_t) using the Hinge Loss function:

$$\ell_t = \max(0, 1 - y_t(\mathbf{w}^T \mathbf{x}_t))$$

- If the loss $\ell_t = 0$, the prediction was correct and safe. The model is **Passive**: $\mathbf{w}_{t+1} = \mathbf{w}_t$.
- If $\ell_t > 0$, the model is **Aggressive**. It updates its weight vector using the following rule:

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \left(\frac{\ell_t}{\|\mathbf{x}_t\|^2 + \frac{1}{2C}} \right) y_t \mathbf{x}_t$$

6.4 Step-by-Step Worked Example

6.4.1 The Dataset

We want to classify incoming news articles as **Sports (+1)** or **Politics (-1)** based on **Word Count** (x_1).

- **Incoming Sample:** $x_1 = 2.0$
- **Actual Label:** $y = +1$ (Sports)

6.4.2 Step-by-Step Calculations

Let's assume our current weight $w_1 = -0.1$ and regularization parameter $C = 1.0$.

1. Compute the current prediction score:

$$z = w_1 x_1 = -0.1 \cdot 2.0 = -0.2$$

2. Compute the Hinge Loss:

$$\ell_t = \max(0, 1 - y \cdot z) = \max(0, 1 - (1 \cdot -0.2)) = \max(0, 1 + 0.2) = 1.2$$

Since loss is greater than 0, the model reacts aggressively!

3. Compute the weight update:

$$\text{Denominator} = \|x\|^2 + \frac{1}{2C} = (2.0)^2 + 0.5 = 4.0 + 0.5 = 4.5$$

$$\text{Step Size} = \frac{\ell_t}{\text{Denominator}} = \frac{1.2}{4.5} \approx 0.267$$

$$w_{\text{new}} = w_{\text{old}} + (\text{Step Size} \cdot y \cdot x)$$

$$w_{\text{new}} = -0.1 + (0.267 \cdot 1 \cdot 2.0) = -0.1 + 0.534 = 0.434$$

6.4.3 Plain English Interpretation

Our initial model incorrectly classified the sports article as politics ($z = -0.2$). Because of this mistake, the Passive-Aggressive algorithm immediately updated the weight from a negative value (-0.1) to a positive value (0.434), ensuring a correct classification on the next attempt.

6.5 Python Implementation & Code Walkthrough

```
1 from sklearn.linear_model import PassiveAggressiveClassifier
2 import numpy as np
3
4 # 1. Initialize the Passive-Aggressive Classifier
5 pa_model = PassiveAggressiveClassifier(C=1.0, max_iter=1000)
6
7 # Create an example dataset
8 X = np.array([[2.0], [0.5]])
9 y = np.array([1, -1])
10
11 # 2. Fit the model to our training data
12 pa_model.fit(X, y)
13
14 # 3. Make a prediction on a new data point
15 print("Prediction:", pa_model.predict([[1.8]]))
```

6.5.1 Under the Hood Code Breakdown

- `max_iter=1000`: Sets the maximum number of times the model will pass over the training data during optimization.
- `pa_model.fit(X, y)`: Internally loops through each sample, applying the passive-aggressive update rule whenever a prediction is incorrect or falls within the margin boundaries.

7 Stochastic Gradient Descent (SGD) Classifier

7.1 What is it? (The Intuitive Definition)

SGD is not a standalone classifier; it is an **optimization framework**. It provides a highly efficient method for training classifiers on massive datasets by evaluating and updating weights using one random sample at a time, rather than waiting to process the entire dataset.

7.2 The Analogy: A Blindfolded Hiker

Imagine you are a blindfolded hiker standing on a mountain during a heavy rainstorm, trying to find your way down to the lowest point of the valley (the minimum loss).

- **Batch Gradient Descent:** You walk around the entire mountain peak, mapping the slope in every direction before taking a single, carefully calculated step downward. This approach is highly accurate but takes a long time.
- **Stochastic Gradient Descent (SGD):** You feel the slope under one foot, immediately take a quick step in that direction, and repeat. Your path down the mountain will be bumpy and uneven, but you will reach the bottom incredibly fast.

7.3 The Mathematical Concept

Instead of calculating the average gradient across all N data points before updating our parameters:

$$\mathbf{w}_{\text{new}} = \mathbf{w}_{\text{old}} - \eta \left(\frac{1}{N} \sum_{i=1}^N \nabla \text{Loss}_i \right)$$

SGD selects a single random data point (i) and immediately updates the parameters based on that point's gradient:

$$\mathbf{w}_{\text{new}} = \mathbf{w}_{\text{old}} - \eta \nabla \text{Loss}_i$$

7.4 Step-by-Step Worked Example

7.4.1 The Dataset

We want to classify emails as **Spam (1)** or **Safe (0)** using the feature **Contains "Viagra"** (x_1).

- **Selected Sample:** $x_1 = 1.0$ (Contains word)
- **Actual Label:** $y = 1$ (Spam)

7.4.2 Step-by-Step Calculations

Let's assume our current model has a weight $w_1 = 0.5$, learning rate $\eta = 0.2$, and uses the Squared Error loss function $\text{Loss} = \frac{1}{2}(y - \hat{y})^2$.

1. Compute the current prediction:

$$\hat{y} = w_1 x_1 = 0.5 \cdot 1.0 = 0.5$$

2. Compute the error gradient with respect to weight w_1 :

$$\nabla \text{Loss} = -(y - \hat{y}) \cdot x_1 = -(1 - 0.5) \cdot 1.0 = -0.5$$

3. Update the weight:

$$w_{\text{new}} = w_{\text{old}} - (\eta \cdot \nabla \text{Loss}) = 0.5 - (0.2 \cdot -0.5) = 0.5 + 0.1 = 0.6$$

7.4.3 Plain English Interpretation

By analyzing a single random email containing “Viagra”, the SGD algorithm identified a prediction error and immediately increased the weight for that feature from 0.5 to 0.6, completing the update in a fraction of a millisecond.

7.5 Python Implementation & Code Walkthrough

```
1 from sklearn.linear_model import SGDClassifier
2 import numpy as np
3
4 # 1. Initialize the SGD Classifier
5 # Using 'log_loss' configures the model to run Logistic Regression
6 sgd_model = SGDClassifier(loss='log_loss', learning_rate='constant', eta0
7                             =0.1)
8
9 # Create an example dataset
10 X = np.array([[1.0], [0.0]])
11 y = np.array([1, 0])
12
13 # 2. Fit the model to our training data
14 sgd_model.fit(X, y)
15
16 # 3. Predict the class of a new sample
17 print("Predicted Class:", sgd_model.predict([[0.9]]))
```

7.5.1 Under the Hood Code Breakdown

- `loss='log_loss'`: Configures the SGD optimizer to minimize log-loss, turning the model into a Logistic Regression classifier. Changing this to `loss='hinge'` would implement a Linear SVM.
- `learning_rate='constant', eta0=0.1`: Instructs the optimizer to use a constant step size of 0.1 for every update.

8 Linear Discriminant Functions

8.1 What is it? (The Intuitive Definition)

A Linear Discriminant Function is a scoring formula used to classify data. If you have multiple target classes, you create a scoring formula for each class. When evaluating a new sample, you plug its features into all formulas and assign the sample to the class that yields the highest score.

8.2 The Analogy: The Video Game Scoring System

Imagine you are playing a video game that evaluates your playstyle at the end of each round to classify you into one of three roles: **Warrior**, **Mage**, or **Rogue**.

The game uses three simple formulas:

- **Warrior Score** = $2.0 \cdot (\text{Swords}) + 0.1 \cdot (\text{Spells})$
- **Mage Score** = $0.1 \cdot (\text{Swords}) + 3.0 \cdot (\text{Spells})$
- **Rogue Score** = $1.0 \cdot (\text{Swords}) + 1.0 \cdot (\text{Spells})$

At the end of a round, if you cast many spells but rarely used a sword, the **Mage Score** formula will output the highest value, and the game will classify you as a Mage.

8.3 The Mathematical Concept

For a multi-class problem with K classes, we define a discriminant function $g_k(\mathbf{x})$ for each class:

$$g_k(\mathbf{x}) = \mathbf{w}_k^T \mathbf{x} + b_k \quad \forall k \in \{1, \dots, K\}$$

We evaluate all functions for a given input vector $\mathbf{x}$, and select the class (C_i) that produces the maximum value:

$$\hat{y} = \arg \max_k g_k(\mathbf{x})$$

8.4 Step-by-Step Worked Example

8.4.1 The Dataset

We want to classify a document into **Sports (Class 1)**, **Finance (Class 2)**, or **Tech (Class 3)** based on the frequency of the word “market” ($x_1 = 3.0$).

8.4.2 Step-by-Step Calculations

Let’s define our discriminant functions for each class:

- **Sports:** $g_1(x) = -1.5 \cdot x_1 + 0.0$
- **Finance:** $g_2(x) = 2.0 \cdot x_1 - 3.0$
- **Tech:** $g_3(x) = 0.5 \cdot x_1 + 1.0$

Let’s compute the scores:

1. **Sports Score:** $g_1(3.0) = -1.5 \cdot (3.0) + 0.0 = -4.5$

2. **Finance Score:** $g_2(3.0) = 2.0 \cdot (3.0) - 3.0 = 6.0 - 3.0 = 3.0$

3. **Tech Score:** $g_3(3.0) = 0.5 \cdot (3.0) + 1.0 = 1.5 + 1.0 = 2.5$

Compare the scores:

$$\arg \max(g_1, g_2, g_3) = \arg \max(-4.5, \mathbf{3.0}, 2.5) = \text{Class 2 (Finance)}$$

8.4.3 Plain English Interpretation

Because the word “market” appeared frequently, the Finance formula produced the highest score (3.0), leading the model to classify the document as a Finance article.

8.5 Python Implementation & Code Walkthrough

```
1 import numpy as np
2
3 class MultiClassLinearDiscriminant:
4     def __init__(self):
5         # Define weights and biases manually for our three classes
6         self.weights = {
7             'Sports': np.array([-1.5]),
8             'Finance': np.array([2.0]),
9             'Tech': np.array([0.5])
10        }
11        self.biases = {
12            'Sports': 0.0,
13            'Finance': -3.0,
14            'Tech': 1.0
15        }
16
17    def predict(self, x):
18        scores = {}
19        # Calculate the score for each class formula
20        for class_name in self.weights:
21            scores[class_name] = np.dot(x, self.weights[class_name]) + self
22            .biases[class_name]
23
24        # Select and return the class name with the highest score
25        return max(scores, key=scores.get)
26
27 # Test the classifier with a sample
28 classifier = MultiClassLinearDiscriminant()
29 sample = np.array([3.0])
30 print("Predicted Class Category:", classifier.predict(sample))
```

8.5.1 Under the Hood Code Breakdown

- `scores[class_name] = ...`: Iterates through our dictionary of formulas, calculating a raw score for each class.
- `max(scores, key=scores.get)`: Finds and returns the dictionary key (class label) associated with the highest computed score.

Algorithm Quick Reference Summary

Table 1: Overview of the 8 core linear classification algorithms.

Algorithm	Analogy	Output Type	Best Used For
Perceptron	Strict Bouncer	Hard Binary	Simple binary classification tasks
Logistic Regression	Weather Forecaster	Probabilistic (%)	When confidence levels are required
Linear SVM	Highway Construction	Margin / Distance	Maximizing decision safety and margin
Fisher's LDA	Flashlights & Shadows	1D Projection	Statistically separating groups
Ridge Classifier	Cautious Investor	Penalized Discrete	Preventing overfitting on noisy data
Passive-Aggressive	Driving Instructor	Real-time Update	Streaming data and continuous learning
SGD Classifier	Blindfolded Hiker	Flexible Loss Output	Massive scale data matching
Discriminant Functions	Video Game Scoring	Multi-class Selection	Quick score-based multi-class classification